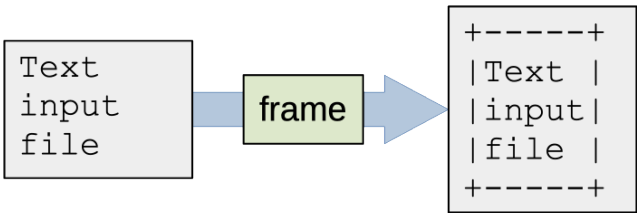
Exercise 13

Dynamic Memory Allocation

On our Moodle homepage, you'll find a source file, **frame.c**, along with sample input files and expected output files for three test cases. You can also download these using the following shell command if you're logged in on remote.eos.ncsu.edu:

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise13/* .
```

The following figure shows what the frame.c program is expected to do. It's given the name of a text file as a command-line argument. It reads all the lines of the text file into dynamically allocated strings, finds the length of the longest line and then prints out the text inside a rectangular box.



Completing the Implementation

To complete your implementation of the frame.c program, you will need to fill in two functions and add some code to main(). You can add more functions if you want to break up your implementation a little bit more.

- char *readLine(FILE *fp)

  This is the most interesting function in your program. Its job is to read the next line of text from the given input file and store it in a dynamically allocated string. It should return a pointer to the start of the string if it is successful, or it should return NULL if there are no more lines of text in the input. You can assume a line always has line termination (a newline character) marking the end. The input file won't have extra characters between the last line terminator and the end-of-file.

  As it reads, the readLine() function will store the characters from an input line in a resizable array of char. Start wth a dynamically allocated array with room for 5 characters. As you read the current line, you may run out of room and need to allocate a larger array. When this happens, follow the usual practice of doubling your array capacity. You can do this the easy way, with realloc(), or you can allocate a new array yourself, copy the contents over and then free the old array. Either way is fine.

  After successfully reading a line be sure to put a null terminator at the end. The rest of your program will be expecting readLine() to return a string. Be sure to allow for the null terminator in your resizable array. You'll need room for all the characters in the current line, plus one more element to hold the null terminator.

- int readText(FILE *fp, char *text[])

  This function should call readLine() repeatedly, storing a pointer to each line of the input file in an element of the given text[] array. When you reach the end of the input, readLine() should return NULL, so you can tell you have all the lines in the file. The capacity of the text[] array is limited. If you're given an input file with too many lines, your readText() function should print the following message to standard error and then terminate with an exit status of 1.

  ```
  Input file too long
  ```

In the main function, you'll add code to open the input file given on the command line. Be sure to close the file when you're done reading. If you can't open the file, your program should print the following message to standard error and then terminate with an exit status of 1. In this error message, *filename* should be the name of the file given on the command line.

```
Can't open file: filename
```

You'll use the readText() function to read the contents of the file into memory, storing a pointer to each line in an element of the **text[]** array. You'll need to determine the length of the longest line in the input; then you can print out the contents of the file with a box around it made of '+', '-' and '|' characters, like in the figure above. For shorter lines, you'll need to print some extra spaces at the end to make sure the vertical bars on the right are all in the same output column.

Before exiting, be sure to free all the heap memory that was dynamically allocated as you were reading the lines of the text file. The text[] array itself is in stack memory, so you won't need to free it, but the string used to store each line will be in heap memory, so you'll need a little loop to free each one.

Testing your Program

When you're done, your program will be able to read any text, up to 1024 lines in length. You should be able to run it as follows to try it out on the first test case.

```
$ ./frame input-1.txt
+-----+
|seven|
|six  |
|five |
|four |
|three|
|two  |
|one  |
+-----+
```

To make sure your program is producing exactly the right output, you can save your output to a file, then check your program's exit status to make sure it ran successfully:

```
$ /frame input-1.txt > output.txt
$ echo $?
0
$ diff output.txt expected-1.txt
```

The **input-2.txt** file is larger, with some blank lines and some lines that have extra whitespace at the start. You can try it out against the expected output to make sure your program is working right:

```
$ ./frame input-2.txt > output.txt
$ echo $?
0
$ diff output.txt expected-2.txt
```

The **input-3.txt** file is a test for error handling. It has too many lines in the input file, so your program should print an error message and exit unsuccessfully. The sample execution below shows capturing the standard error output to a file. After checking the program's exit status (should be unsuccessful), we check the error message against an expected error message:

```
$ ./frame input-3.txt 2> stderr.txt
$ echo $?
```

```
1
$ diff stderr.txt error-3.txt
```

The starter doesn’t include an input-4.txt file, but it does have an error message for this test case. You can use this to check your program behavior when it can’t open the given input file:

```
$ ./frame input-4.txt 2> stderr.txt
$ echo $?
1
$ diff stderr.txt error-4.txt
```

Submitting your Solution

When you're done, submit the source, frame.c to the exercise_13 assignment on Moodle.